

UNIFORMING DEVELOPMENT ENVIRONMENTS

Simple Workshop Setups with Devcontainers & Codespaces

viadee 
IT-Unternehmensberatung

Agenda

- Motivation
- Codespaces
 - Prerequisites
 - Benefits / Concepts / Features
 - Demo
 - Challenges / Limits
 - Other Use Case Examples
- Alternative Tools

**„START CODING IN
SECONDS“**



viadee Consulting AG

US IN NUMBERS:

-  **250+** qualified colleagues
-  **3** Locations: Cologne, Münster & Dortmund
-  **100%** Equity financing
-  **3.000** days/year research and development

+

OUR CULTURE:

-  Curiosity & Openness
-  Regionality & Short ways
-  Training & Development
-  Participation & Appreciation

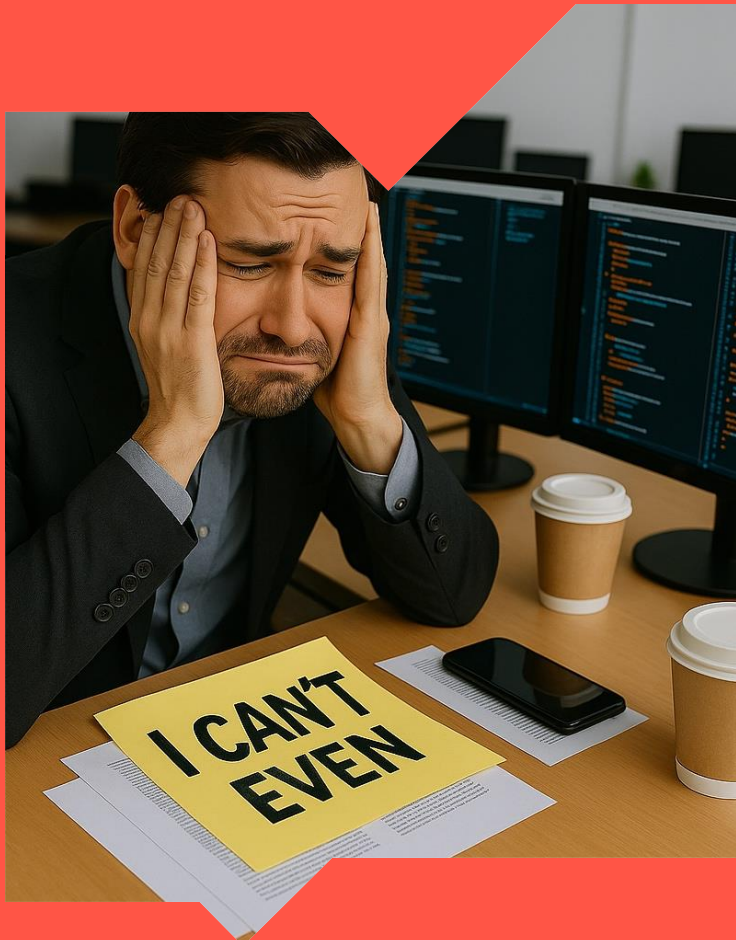




1

Motivation

What's the problem?



Motivation

Always had a few problems during workshops:

- Individual setup problems
- Docker / Kubernetes
 - Since Docker Desktop's license
→ Setup even more diverse
 - Diverse hardware, diverse software
 - Sharing Cluster Credentials
- Single-Day-Workshops should not end up setting up stuff

2 Codespaces



Why Codespaces? Setup, Features, Challenges

Codespaces

Prerequisites

Who can use Codespaces?

- Github only, as part of Organization
- Has to be enabled (and configured) via Organization-Admins
- Guests need access to this internal Organization repo

How to set up a codespace?

- Create new repo or use existing
- Give users write permissions in this repo

Codespaces

Benefits

Whats the „magic“ of Codespaces?

- Same setup, same basis
- Focus more on coding / playing around with tools to learn
- “Couldn’t care less” about setting up dev environment
- Setup problems can be fixed before the workshop
- Automatically install CLI tools of the workshop topic
- Participants can immediately start coding
- Pay per „use“ → Automatic cleanup + free contingent

Codespaces

Concepts / Features

What's behind the curtain?

- Small VM, hosted in Cloud, maintained by Github
- Automatic creation + Management + Entry via Git-Repo
- Starts a container including VS Code + current Git-Repo
- Click through configuration → Creates `.devcontainer/devcontainer.json`

Not enough? It's customizable!

- **Either:** Big pool of base images (Debian / Java / ...)
- **Or:** Custom Dockerfile
- **Either:** Big pool of „features“ / CLI tools to be automatically installed
- **Or:** Run custom scripts on creation (**or both!**)

Codespaces

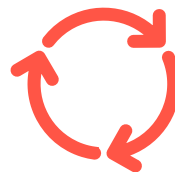
Concepts / Features

Workflow configuring Codespaces

1. Create Codespace

2. Iteratively:

- 1. Cmd/Ctrl + Shift + P → „Codespaces: Add Dev Container Config“ → Modify
- 2. Click through configs
- 3. Cmd/Ctrl + Shift + P → „Codespaces: Rebuild“



3. Commit + Save for future Codespaces

3 **Quit the drool, show me the tool**



Demo

Codespaces

Concepts / Features

```
1 // For format details, see https://aka.ms/devcontainer.json. For config options, see the
2 // README at: https://github.com/devcontainers/templates/tree/main/src/debian
3 {
4   "name": "Debian",
5   // Or use a Dockerfile or Docker Compose file. More info: https://containers.dev/guide/dockerfile
6   "image": "mcr.microsoft.com/devcontainers/base:bullseye",
7   // Features to add to the dev container. More info: https://containers.dev/features.
8   "features": {
9     "ghcr.io/devcontainers/features/kubectl-helm-minikube:1": {
10       "version": "latest",
11       "helm": "3.19.1",
12       "minikube": "none"
13     },
14     "ghcr.io/audacioustux/devcontainers/kustomize:1": {},
15     "ghcr.io/jajera/features/gcloud-cli:1": {}
16   },
17   "postCreateCommand": "chmod -R 777 ${containerWorkspaceFolder}/.devcontainer/scripts && bash ${containerWorkspaceFolder}/.devcontainer/scripts/post-create.sh",
18   // Use 'forwardPorts' to make a list of ports inside the container available locally.
19   // "forwardPorts": [],
20   // Configure tool-specific properties.
21   "customizations": {
22     // Configure properties specific to VS Code.
23     "vscode": {
24       // Add the IDs of extensions you want installed when the container is created.
25       "extensions": [
26         "ms-kubernetes-tools.vscode-kubernetes-tools",
27         "redhat.vscode-yaml",
28         "tim-koeher.helm-intellisense"
29       ]
30     }
31   }
32   // Uncomment to connect as root instead. More info: https://aka.ms/dev-containers-non-root.
33   // "remoteUser": "root"
34 }
```

Useful additional Features

- Copilot
- Port-Forwarding
- Auto-Installation of Extensions
- Set env vars for Codespaces
- Running custom “post create” scripts
 - Git clone public repos
 - Bash Auto-Completions
 - Env vars → cluster connection

Codespaces

Challenges / Limits

☹️ But I haven't set it up on my machine now! I can't use the learned stuff!

- **Yes, but...**

- Experience showed, participants need to take time after workshops anyway
- Very motivating → More time using the tools, less time dealing with errors / setup initially
- Setup problems usually occur only for a few people
- Enhances the exchange with the trainers in the days/weeks after the workshop
- The more you understand the tool and concepts, the more likely you are capable of solving setup problems on your own

☹️ All my progress of the workshop is gone!

- **Yes, but...**

- It's a Git-Repo! 😊
- → Push to other origin / push & download branch as zip / copy files from Codespace to disk (locally)

Codespaces

Challenges / Limits

☹️ It needs an eternity to start, if I need more than only one CLI tool!

- **Yes, but...**

- It's possible to build and save „Pre-Built Container Images“ for your Codespace
 - Extra costs → Reduce high availability if not needed
- Keep in mind: The complexer the setup, the more you profit from it
 - New co-workers are super easy and quick ready to work

☹️ The “features” / tools throw errors on initialization!

- **You can try...**

- Using own scripts installing the needed tools / setting up env vars
- Look up the code of the „feature“

☹️ I don't want to use a web-based IDE!

- Devcontainer-Config is compatible with local VS Code

Codespaces

Other Use Case Examples

When does using Codespaces in everyday work makes sense?

- Many **context switches / multiple projects**
 - Based on same/similar tooling, but different versions (e.g. Java/Maven or Node/NPM)
- In projects, that are **rarely touched** but need some initial setup to work
→ e.g. (Java-/Maven-)Demo-Apps
- On events, where people with very **diverse Hardware & Software** come together and want to quickly be able to code
- When **experimenting with some new tools** without installing everything on your machine
- When working with **thin clients** (e.g. workshop for students / individuals / private persons)
- When **focusing on the doing or on results** is more relevant than making it run on everyone's machine

Alternative Tools

- Devcontainers → Generic framework
- Codespaces is Github's tool to realize Devcontainers
- Other tools: More or less same features, other initialization, other entrypoints
- Not that integrated into Github, but more flexible
 - host on own resources, integrate into internal systems
 - harder to set up in the beginning, more maintainability
- **Alternatives:**
 - Coder (<https://coder.com/>)
 - Ona (aka GitPod in the past) (<https://ona.com/>)
 - AWS CodeCatalyst & Microsoft Dev Box

Thanks for your time



Max Strohbieter
IT-Consultant

✉ max.strohbuecker@viadee.de

in [linkedin.com/in/max-strohb%C3%BCcker-93542b107/](https://www.linkedin.com/in/max-strohb%C3%BCcker-93542b107/)

viadee 
IT-Unternehmensberatung

